Concept Formation by View Application: A Second Domain on Shared Foundations in CogLean

Jeff Shrager

Draft, November 2026

Draft for Comment[1]

Abstract

CogLean is a library of cognitive theory in the Lean proof assistant, designed on the model of Mathlib: a general library of reusable cognitive representations, mechanisms and processes, on which models of any kind can be built, with their consequences both machine-checked and executable. A companion paper introduced it, with multi-digit arithmetic as its first worked example. A general library has to be shown general by use. Here we build on CogLean in a very different area of cognition: how people form concepts of something new by applying concepts they already have. We formalize view application (Shrager, 1987), in which a learner understands a novel device or domain by merging a prior schema, a *view*, into a current theory, and we add the two things it lacked: an account of which view is brought in, and of when the theory is revised.

Both additions are built from general CogLib components that the arithmetic example also used. Which view is brought in is decided by retrieval through the same memory interface, so EPAM-style and ACT-R-style memories can be exchanged, and their proved laws apply. When the theory is revised is decided by the same decision cycle that runs the arithmetic solver: a failed prediction is an impasse, and applying a view is the repair. We prove results at the levels the companion paper distinguished. In general, for any theories, actions and revision processes, a failed prediction is exactly an impasse and a correct theory is never revised. For view retrieval, a cue confused with another brings in the other's view, and the confusion-error pattern first stated for arithmetic holds again, through a different mechanism. For a class of memories, every memory with the law of determinacy makes a learner who keeps consulting one cue keep applying the same failed interpretation, a formal account of perseveration. Views can also be composed from parts (a key enters a number; the number is a time, or a duration), and a revision can then replace one part and keep the rest. Because views re-interpret types, we prove that no theory in which two keys share a type can tell them apart, which makes *differentiation*, splitting a type, necessary rather than optional; we prove that a split by itself changes no prediction, and show that a learner who splits when a revision would undo an explanation learns both keys, while one who cannot split never does and oscillates between interpretations. Earlier view-application results in the library show the same process removing over-extension from number concepts while leaving typicality unchanged. A worked case, learning a microwave oven, gives concrete witnesses; its exact values are in an appendix. The point is not the microwave but the reuse: a different domain, much of the same library, and phenomena that transfer across domains.

[1]This draft has *not* been carefully checked by the author. It is likely to contain erroneous citations and details. Think of this as an over-starched blog post. I'm happy to receive constructive feedback at: `jshrager@gmail.com`

1 Introduction

The companion paper [1] introduced CogLean [2], a library of cognitive theory in the Lean proof assistant, organized on the model of Mathlib. Its claim was methodological. If mechanisms are defined once, phenomena are stated as predicates independent of any mechanism, and the consequences of theories are proved rather than simulated, then theoretical results can accumulate. Theories can be composed, their theorems reused, and new consequences derived from old ones. The demonstration composed EPAM's recognition [3], Siegler and Shrager's distributions of associations and a problem solver into a model of three-digit addition.

CogLean was not designed for arithmetic. It was designed as a general library: representations, mechanisms and processes defined once, on which any model can be built, with its consequences both proved and executable. Arithmetic was the first example worked out in it. One example cannot show that the design succeeded; a second, in a very different domain, is a test of it. This paper builds a model in a domain as different from arithmetic as we could find within the library's present reach: the formation of concepts and theories, and in particular how people make sense of something new by applying concepts they already have.

That process has been described in many ways. Concepts are embedded in theories that give them coherence [4]. Children's concepts change by restructuring, not only by accretion [5, 6]. Analogy maps the structure of a known domain onto a new one [7]. Prior schemas or frames supply defaults and expectations [8]. People learning to operate a device build mental models of it [9, 10]. Shrager's account of *view application* [11] is a computational version of one part of this story. A learner exploring a device without instructions [12] holds a theory of it. The learner changes that theory by applying a *view*: a schema of a familiar kind of thing, such as "a device that stores a program and runs it", which is merged into the theory, overriding it where they conflict.

The 1987 account left two things open. It had no principled account of which view a learner brings in, and it described, but did not formalize, when the theory is changed. Both turn out to be supplied by general components CogLib already had, which the arithmetic example had also used.

1. **Which view: retrieval through a shared memory interface.** A cue (a key's label, a feature of the display) is coded and recognized by a memory, and the view stored with what it calls up is applied. The memory is CogLib's *retriever* interface, the one that let the arithmetic model exchange EPAM's discrimination net for ACT-R's declarative memory. The laws of memory proved for those families apply unchanged.

2. **When to revise: the decision cycle.** The arithmetic model's backup was a problem solver on CogLib's decision cycle, with impasses and repairs in the tradition of repair theory and Soar [13, 14]. Revising a theory at a failed prediction has the same shape. The failed prediction is an impasse, and applying a view is the repair.

Section 2 summarizes view application as formalized in CogLib, with the concept results it already supports. Section 3 adds retrieval of views, and Section 4 the revision loop. Section 5 presents the phenomena, including one that transfers from arithmetic. Section 6 composes views from parts and adds differentiation. Section 7 states what was shared and what was new. Section 8 gives a worked case, qualitatively. Section 9 discusses what this does and does not show. As in the companion paper, the general theorems are the result; the numerical values at an illustrative setting (Appendix B) are witnesses that the hypotheses can be met and that memories differ.

2 View application in CogLib

Knowledge nets. CogLib represents knowledge as semantic nets whose nodes are *terms*: atoms, structured values, and typed holes. A frame is an atom with links. A *type* frame has a kind and no parent; an *instance* frame has a parent. Slot values are terms, and patterns, which are terms with holes, say what an instance's slot holds. The same representation carries the library's frames, prototypes, categorization methods and semantic memory, so view application operates on the representation other packages use, not on one of its own.

The process. Following Shrager [11], applying a view to a theory (`CogLib/Process/ViewApplication.lean`) proceeds in five steps:

1. copy the view;
2. merge the theory into it, matching each theory type frame to the view's frame of the same abstract type or, failing that, to an unused view frame of the same kind, at random;
3. coerce the slots of re-parented instances from their old type's patterns to their new type's;
4. bind the view's holes to theory atoms;
5. give each object a single function.

Ambiguity is resolved at random and never revisited, so application gives a probability distribution over theories. CogLib proves, for any view, theory and matching, that the view is kept whole (`merge_keeps_view`); that where both speak of a slot of a matched frame, the view wins, a non-monotonic change (`merge_view_wins`); that unmatched theory frames survive, renamed (`merge_keeps_unmatched`); and that instances of a matched type are re-parented (`merge_reparents`). These are properties of the process, not psychological claims, but they say exactly which parts of a theory a view can change.

Two results already in the library. The `ViewApplication` package supplies data for two domains, and the process is CogLib's.

- **BigTrak.** Applying the 1987 paper's CODE.AND.STARTUP view to its initial theory of the BigTrak toy reproduces the paper's Table 4 exactly, with probability 1/2; the other half of the time a tie is broken the other way, as the paper's random tie-breaking implies (`table4_reproduced`).
- **Number concepts.** Concepts such as *even* and *multiple of 3*, learned as prototypes and combined by selective modification [15] with graded membership [16], over-extend: 14 is an *even multiple of 3* to degree about .80 [cf. 17]. Applying an ARITHMETIC view replaces the categorization method of the concept frame. Afterwards, *even multiple of 3* is exactly divisibility by 6 for every number (`after_exact`), there is no over-extension at any number, and typicality is unchanged for every number (`typicality_unchanged`). So typicality and membership dissociate: 14 is more typical of *even multiple of 3* than 9 is of *multiple of 3*, yet only 9 is a member (`dissociation`). That is the pattern Armstrong et al. [18] found for *odd number*: graded typicality for a concept whose membership is all-or-none. Before the view, it is impossible, because membership is then a nondecreasing function of typicality (`Cog.not_dissociates`).

The second result is a concept-formation result in its own right: a change of theory, not of examples, turns a prototype concept into a definitional one while preserving its graded structure.

3 Which view? Retrieval through a shared memory

Two selectors. The library offers two mechanisms for choosing a view, both general.

- **Spreading activation** [19] over the net that holds the learner's theory and the views' associations (`ViewApplication/Selection.lean`). The same cue brings in different views depending on what the learner already knows: a "+" key is read as adding with probability 2/3 by a learner whose theory has no memory, but as appending a program step with probability 3/5 by a learner whose theory already has one (`naive`, `primed`).
- **Retrieval** (new, `CogLib/Process/ViewRetrieval.lean`). A cue is coded as features and recognized by a memory, given as any `retrieve`, and the view stored with what is called up is applied. If nothing is called up, the theory is unchanged.

Why retrieval matters. Retrieval goes through the same interface as the arithmetic model's recognition of problems. So the memory is a parameter, and what is proved about memories applies. Three results follow, for any memory, cue coding, views and theory:

- a cue confused, for certain, with another brings in exactly the other's view (`confusion_gives_neighbours_view`);
- with a memory that has the law of *determinacy*, which says that retrieval in any reachable state calls up a single outcome, each cue brings in one view, the same at every attempt and for every theory (`view_of_determinate`);
- the confusion-error pattern holds for view retrieval, given conditions (Section 5).

The laws are those of the companion paper's memory matrix, which proves that EPAM's discrimination net is determinate at every setting and that ACT-R's declarative memory [20] is not determinate at any setting.

4 When to revise: theory change as problem solving

The loop. An instructionless learner acts, predicts the response with its current theory, and changes the theory when a prediction fails [11, 12]. CogLib states this as an instance of its decision cycle (`CogLib/Process/TheoryRevision.lean`), not as a new control structure:

Learning by revision	Decision cycle
the theory, the memory, the episodes still to explain	the state
accept the theory's prediction	operator `confirm`, whose precondition is that it is right
a failed prediction	an impasse
revise the theory, prompted by a cue	a repair operator
which cues to consult	repair heuristics
not consulting a cue again for the same failure	a critic

What the loop assumes. The loop's definitions assume only actions and observed responses. A learner is a `Reviser`, which supplies three things: how a theory predicts the response to a sequence of actions; how a cue revises a theory, which may be any process, with any memory, giving a distribution; and which cues an episode offers:

```
structure Reviser (Θ M Act Resp C B : Type) where
  predicts : Θ → List Act → Option Resp
  revise   : C → Θ → M → Dist (Θ × M × B)
  cues     : List Act × Resp → List C
  once     : Bool
```

Nothing in the loop mentions knowledge nets, views or devices. Two results hold for every reviser: the learner is at an impasse exactly when the current theory's prediction of the next episode fails (`impasse_iff_fails`), and a theory that predicts every episode is never revised (`correct_theory_stable`). The first is close to the definitions; the second is the formal content of "if it isn't broken, the learner doesn't fix it", and it holds whatever the revision process is.

View application as one reviser. The view-application learner is one instance (`CogLib/Process/ViewRevision.lean`): revision is view retrieval followed by view application, and memory is not changed by revising. Its theories are exactly those of view retrieval (`viewReviser_theory`). How a knowledge-net theory predicts is a further, separate choice (`CogLib/Process/TheoryExecution.lean`): each action is a frame whose function slot's denotation acts on a state, in sequence. That is one model of acting, sequential and deterministic, and the loop does not depend on it. Other revisers can be added and inherit the loop's results: rule induction [21], for example, or search in a hypothesis space guided by experiments [22].

5 Phenomena, old and new

5.1 A phenomenon that transfers: confident misinterpretation

The companion paper stated the *confusion-error pattern* as a predicate over answer and strategy-route distributions, separate from any mechanism:

```
def ConfusionErrorPattern trial wholeWrong isWhole isBackup x q : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧
  prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧
  prob (trial x) isBackup = prob (trial q) isBackup
```

In arithmetic, x is a problem confused with a familiar q. Retrieval never errs on q, every retrieved answer to x is wrong, and x falls back on the backup exactly as often as q.

For view retrieval, read a cue's "answer" as the prediction its revised theory makes, and read exploration (no view retrieved) as the backup. Then the same predicate holds (`view_confusion_pattern`), under stated conditions:

- q and x both call up the same stored code y, for certain;
- y's view, applied to the theory, predicts q's response correctly and x's wrongly on every outcome of the application.

The learner who takes one key for another applies the other's view with full confidence and predicts wrongly, with no more exploration than the familiar case.

The mechanism is different. There is no associative search with a confidence criterion, as in arithmetic, only recognition followed by application. We did not force view retrieval into the arithmetic mechanism, because that would have required inventing a criterion search over sampled predictions that view application does not perform. What transfers is the *phenomenon*, which is

exactly what stating phenomena independently of mechanisms is for. The conditions carry much of the content here, and the result is close to its premises; its interest is that the predicate, written for arithmetic, needed no change.

5.2 New phenomena: garden paths and perseveration

CogLib states two phenomena of theory change over the sequence of interpretations a learner brings in at successive failures (`CogLib/Phenomena/TheoryChange.lean`):

- a *garden path*: the first interpretation adopted is later replaced;
- *perseveration*: the same interpretation is adopted at two successive failures.

Both are mechanism-independent predicates; as with the confusion-error pattern, applying them to people requires identifying which interpretation a learner has adopted, for example from verbal protocols [12].

Perseveration, for a class of memories. Combining the loop with the law of determinacy gives a class-level result (`revise_same_of_determinate`). For every memory with the law, a view-application learner revising by a given cue brings in one and the same retrieval at every attempt and from every theory. Two conditions apply: the memory is in a state reachable by study, and it is not changed by revising. A learner who keeps consulting only that cue keeps re-applying a failed interpretation. This is a formal relative of fixation and *Einstellung* [23]: the persistence follows from a property of memory, not from anything about the device or the view. A memory without the law, such as ACT-R's, can retrieve different views from the same cue, and escape. The result says the law is sufficient for perseveration in this loop; it does not say the law is necessary, or that people's memories have it.

6 Composing views, and differentiation

Views from parts. A view such as SET-A-TIME bundles two ideas: that a key enters a number, and that the number is a time of day. Decomposed, they are separate views: SET-A-NUMBER, and NUMBER-IS-A-TIME or NUMBER-IS-A-DURATION, each mentioning one slot of the key's type. View application composes them because its merge lets a view win only on the slots it mentions; every other slot, including one an earlier view supplied, survives (`merge_keeps_unmentioned`, `CogLib/Process/ViewComposition.lean`). In the microwave, no single part makes a prediction; any two of the right kind do, in either order; and replacing NUMBER-IS-A-TIME by NUMBER-IS-A-DURATION keeps "the key enters a number" (`one_view_not_enough`, `two_views_compose`, `revision_keeps_number`). A revision can thus change part of an interpretation and keep the rest: tuning rather than restructuring, in Rumelhart and Norman's terms [6]. That views on disjoint slots commute in general is open; the instances are certified.

Why differentiation is needed. Views re-interpret *types*: every instance of a matched type changes together. When predictions read an action's type only through its slots, two actions of one type are read alike (`same_parent_same_reading`, `CogLib/Process/TypeSplit.lean`). In the microwave this becomes an impossibility for *every* theory: if TIME COOK and CLOCK share a type and both enter a number, 9 0 START is predicted alike after either, so no such theory predicts both cooking for 90 seconds and an invalid time (`never_both_shared`). No choice of view, and no amount of revision, can fix this. The concept must split.

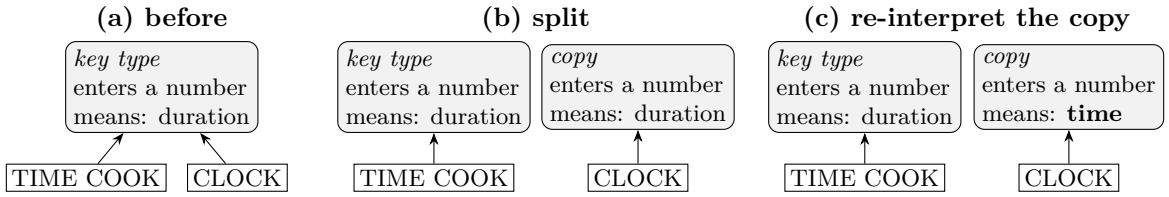


Figure 1: Differentiation in the microwave. (a) Both keys share a type, so they are read alike, and no view on that type can make both right (`never_both_shared`). (b) Splitting copies the type for CLOCK; nothing predicts differently yet (`split_conservative`). (c) NUMBER-IS-A-TIME, aimed at the copy, changes CLOCK only (`split_separates`).

Splitting a type. `splitType` copies an action's type, with the same kind and slots, and re-parents the action to the copy; `retarget` aims a view at a chosen type. Two general results: a split by itself changes no prediction (`split_conservative`), and it separates the action from the other instances of its old type (`split_separates`). Differentiation is therefore conservative: it does not change what the learner believes, only what it can come to believe. This matches the accounts of conceptual change in which one undifferentiated concept becomes two [5, 24].

A split, step by step. Figure 1 follows the microwave learner. After the first episode, TIME COOK and CLOCK are both instances of one operation type, which enters a number and says the number is a duration (panel a). CLOCK's episode then fails: the theory predicts cooking, and the device reports an invalid time. Re-interpreting the shared type as NUMBER-IS-A-TIME would fix CLOCK and break TIME COOK, which the learner has already explained. So the learner first copies the type for CLOCK (panel b). The copy has the same slots, so nothing predicts differently yet (`split_conservative`). It then applies NUMBER-IS-A-TIME to the copy alone (panel c). Now each key has its own type, and both are predicted correctly.

Aiming a view. A view names the type it re-interprets, and view application matches it to a theory type of the same name or, failing that, to one of the same kind. After a split there are two types of the same kind, so a view must be aimed at one of them. `retarget` renames the view's frame to the chosen type. For the name to be decisive, view application now reserves a view frame for the theory frame of the same name, so that another theory frame of the same kind cannot take it first. This is the reading of Shrager's "the view's frame of the same abstract type" [11], and every earlier result (BigTrak, number concepts, view selection) checks unchanged.

What a split costs. A split is conservative, but not free: the theory gains a type, and with it a distinction that later revisions must keep track of. A learner that split freely would multiply types for no reason. The conflict-driven policy below splits only when forced, when the one alternative, re-interpreting the shared type, would undo an explanation.

When to split. The learner (`viewSplitReviser`) splits when re-interpreting the failing action's type would break an episode it has already explained, which is conflict-driven differentiation; otherwise it applies retrieved views to the failing action's type. To support this, the revision loop now records the episodes it has explained, and a learner's cues may depend on them and on the current theory; the loop's theorems are unchanged.

CogLib component	Arithmetic	Concepts	Role here
Exact finite distributions, `bind`, `Same`	yes	yes	view application's random matching; sessions
Retriever interface; EPAM nets; ACT-R memory	yes	yes	which view a cue brings in
Laws of memory (determinacy, certainty, stability)	yes	yes	perseveration for a class of memories
`ConfusionErrorPattern`	yes	yes	confident misinterpretation
Decision cycle with impasses and repairs	yes (solver)	yes (revision loop)	a failed prediction is an impasse
Knowledge nets (semantic nets over terms)	—	yes	theories and views; shared with concepts and semantic memory
View application	—	yes	the revision itself
New: theory execution; view retrieval; the revision loop; view composition; type splitting; garden paths, perseveration, oscillation	—	new	general, in CogLib; no device terms

Table 1: The CogLib components used by the arithmetic model of the companion paper and by concept formation here. The new components are general and live in CogLib; the microwave of Section 8 is data in its own package.

Consequences. With the microwave's episodes TIME COOK, then CLOCK, then TIME COOK again, a learner that cannot split never ends knowing both keys, as the impossibility theorem requires, and every such learner that finishes *oscillates*: the clock's meaning displaces cooking's, and cooking's comes back. A learner that splits on conflict ends knowing both whenever it finishes, and never oscillates (`no_split_session`, `split_session`). Oscillation is stated, like the garden path, as a mechanism-independent predicate over the interpretations adopted for a type (`oscillates`). These results concern representation, not memory, so the sessions use EPAM's net only.

7 What was shared, and what was new

Table 1 lists the CogLib components the two domains use.

Two observations. First, the shared components were reused, not imitated. The same memory definitions, the same law proofs and the same decision cycle are imported, and the results that depend on them (perseveration from determinacy, the transferred pattern) are proved by citing them. Second, the new components went into CogLib in general form. The revision loop needs only actions and responses, and the device appears only as data in a separate package. A third domain that revises theories at failed predictions can use the loop without the view machinery, and one that retrieves schemas can use view retrieval without the loop.

8 A worked case: learning a microwave

The `DeviceLearning` package supplies a generic microwave oven as data. It has keys TIME COOK, CLOCK, the digits 9 and 0, and START. After TIME COOK the digits are a cooking time, so 9 0 START cooks for 90 seconds; after CLOCK they are a time of day, so 90 is invalid. The

	EPAM net	ACT-R memory
Label and display	a garden path when the label is consulted first (SET-A-TIME, then DURATION); always solved	garden paths rarer (recency favours the timer); usually solved
Label only	certain perseveration on SET-A-TIME; never solved	graded retrieval escapes; usually solved
General status	determinate: perseveration follows for every such memory	not determinate at any setting

Table 2: The microwave learner with two memories, qualitatively. The last row is general; the others describe one illustrative setting, given exactly in Appendix B.

learner's initial theory understands the digits and START. It does not know what TIME COOK and CLOCK do, so it predicts nothing, and the learner is at an impasse at once.

Two views are available: SET-A-TIME, in which such a key sets a time of day, and DURATION, in which it sets a cooking time. The learner's prior knowledge, which is illustrative, is a VCR's SET TIME key stored with SET-A-TIME and a kitchen timer stored with DURATION. Cues are the key's label and the display (a countdown, "1:30"). Two memories hold the prior knowledge.

- **EPAM's net** notices the countdown first, so it tests only that. Both key labels call up the VCR's SET TIME, and the display calls up the kitchen timer (`net_facts`).
- **ACT-R's memory** retrieves gradedly: the TIME COOK label calls up the timer, the VCR, or nothing (`act_graded`).

Two learners consult cues: one consults the label and the display, each once per failure, in random order; the other consults only the label.

Table 2 summarizes the outcomes qualitatively.

With EPAM's net, TIME COOK and CLOCK are confused with the VCR's SET TIME. So the confusion-error pattern holds (`epam_confusion_pattern`): CLOCK's episode is always predicted correctly and TIME COOK's always wrongly, with no exploration. The label-only EPAM learner perseverates for certain, as the class-level theorem says it must (`epam_revisions_same`, an instance). Once DURATION is adopted, the theory predicts the episode and is never revised again (`duration_kept`, an instance of `correct_theory_stable`).

9 Discussion

What this shows about cumulativity. The companion paper argued that a library of reusable *results* is a contribution to theory, and a library of reusable code is not. Here the reused results are the laws of memory, the confusion-error pattern and the decision cycle's structure. They were first used in the arithmetic example, and they served unchanged in forming a theory of something new from concepts already held. The new pieces were stated generally, as the library's design requires, so they are available to the next model. A second domain does not establish that the library will scale. It is evidence that its components were cut at general joints, as intended, and not around the first example.

Relation to cognitive architectures. Cognitive architectures such as Soar [14, 25] and ACT-R [20], and the Common Model of Cognition [26], have long made cognitive theory cumulative by

providing shared mechanisms in which models of different tasks are built and integrated; a view-application learner could be built in either. CogLean is complementary to them, not a competitor. What it adds is the accumulation of proved results about mechanisms, which can be reused deductively, and stated across architectural commitments. Both appear here. The decision cycle's results, proved once, hold for theory revision without being re-established, and perseveration is proved for every memory with the law of determinacy, which covers EPAM's net and not ACT-R's declarative memory. That is a statement about a class of memories, not about one architecture. Its limits are those of the companion paper: the law is shown sufficient, not necessary, and nothing here establishes general equivalence or divergence among architectures.

What it says about concept formation. Formalization sharpened three points of the 1987 account.

- **Which prior concept is recruited is a memory question.** With retrieval as the selector, properties of memory become predictions about concept formation: determinacy implies perseveration, and confusion implies confident misinterpretation.
- **Revision as problem solving.** When to revise is not a separate learning mechanism but the impasse structure of problem solving, connecting theory change to the repair-theory and Soar traditions.
- **Views act on types, so concepts must sometimes split.** Because a view re-interprets a *type*, no theory in which two actions share a type can tell them apart. Differentiation is then necessary, and it is conservative: the split itself changes nothing, and only later re-interpretation separates the concepts. Conflict-driven splitting is the difference between oscillating forever between two interpretations and settling on both.

Limitations.

- Memory does not learn during revision: a cue never comes to retrieve a better view. Associating cues with the views that succeeded, or discriminating a cue after an error, is needed.
- Splitting is conflict-driven and, when offered, the only repair; a learner that weighed splitting against overwriting would need weighted repair choice or critics that test a repair's result, which the problem solver does not yet provide. That views on disjoint slots commute is proved only in instances.
- The microwave, its cues and the learners' prior knowledge are illustrative, and the numbers are not predictions about people. No data are fitted.
- ACT-R's memory appears without retrieval as use, as in the companion paper; its values would change.
- Several mechanisms of the 1987 learner are not formalized, among them multiple view sources and combinations of selection by activation with selection by retrieval.
- As in the companion paper, every result is conditional on its premises, and none is an empirical finding.

Future work. The natural next steps are learning in memory; the general commutation law for views; a learner that chooses between splitting and overwriting; a second reviser (rule induction or hypothesis testing) on the same loop, to show that the loop's results transfer between revision processes as the confusion-error pattern transferred between domains; and contact with data, of which the instructionless-learning protocols [12] are the obvious first source.

Conclusion. CogLean was designed as a general, Mathlib-like library of cognitive processes, on which any model can be built with proofs and executability; arithmetic was only its first worked example. Building a model of concept formation on it, the library supplied the memory interface and its laws, a phenomenon first stated in the arithmetic example, and the decision cycle. View application needed only new components that are themselves general. The results were proved at the levels the companion paper set out: for every learner, for every memory with a law, and at a setting as a witness. The library is still small. But this is the behavior a cumulative library should show.

Code. All definitions and theorems are in the CogLean repository [2]: CogLib's `Process/ViewApplication`, `ViewRetrieval`, `TheoryRevision`, `ViewRevision`, `TheoryExecution` and `Phenomena/TheoryChange`; and the `ViewApplication` and `DeviceLearning` packages. Every theorem is fully proved and depends only on Lean's standard axioms.

References

[1] Jeff Shrager. CogLean: Towards a cumulative, machine-checked theory of cognition. Draft for comment, `papers/2026_multidigit/` in the CogLean repository, 2026.

[2] Jeff Shrager. CogLean: A library of cognitive theory in Lean 4. `https://github.com/jeffshrager/CogLean`, 2026.

[3] Edward A. Feigenbaum and Herbert A. Simon. A theory of the serial position effect. *British Journal of Psychology*, 53(3):307–320, 1962. doi: 10.1111/j.2044-8295.1962.tb00836.x.

[4] Gregory L. Murphy and Douglas L. Medin. The role of theories in conceptual coherence. *Psychological Review*, 92(3):289–316, 1985. doi: 10.1037/0033-295X.92.3.289.

[5] Susan Carey. *Conceptual Change in Childhood*. MIT Press, Cambridge, MA, 1985. Unverified.

[6] David E. Rumelhart and Donald A. Norman. Accretion, tuning and restructuring: Three modes of learning. Technical Report ADA030406, Defense Technical Information Center, 1976.

[7] Dedre Gentner. Structure-mapping: A theoretical framework for analogy. *Cognitive Science*, 7(2):155–170, 1983. doi: 10.1016/S0364-0213(83)80009-3.

[8] Marvin Minsky. A framework for representing knowledge. In Patrick H. Winston, editor, *The Psychology of Computer Vision*, pages 211–277. McGraw-Hill, New York, 1975. Unverified.

[9] Dedre Gentner and Albert L. Stevens, editors. *Mental Models*. Lawrence Erlbaum, Hillsdale, NJ, 1983. Unverified (reprinted by Psychology Press, 2014, doi:10.4324/9781315802725).

[10] David E. Kieras and Susan Bovair. The role of a mental model in learning to operate a device. *Cognitive Science*, 8(3):255–273, 1984. doi: 10.1207/s15516709cog0803_3.

[11] Jeff Shrager. Theory change via view application in instructionless learning. *Machine Learning*, 2(3):247–276, 1987. doi: 10.1023/A:1022659315124.

[12] Jeff Shrager and David Klahr. Instructionless learning about a complex device: The paradigm and observations. *International Journal of Man-Machine Studies*, 25(2):153–189, 1986. doi: 10.1016/S0020-7373(86)80075-X.

[13] John Seely Brown and Kurt VanLehn. Repair theory: A generative theory of bugs in procedural skills. *Cognitive Science*, 4(4):379–426, 1980. doi: 10.1207/s15516709cog0404_3.

[14] John E. Laird, Allen Newell, and Paul S. Rosenbloom. SOAR: An architecture for general intelligence. *Artificial Intelligence*, 33(1):1–64, 1987. doi: 10.1016/0004-3702(87)90050-6.

[15] Edward E. Smith, Daniel N. Osherson, Lance J. Rips, and Margaret Keane. Combining prototypes: A selective modification model. *Cognitive Science*, 12(4):485–527, 1988. doi: 10.1207/s15516709cog1204_1.

[16] James A. Hampton. Typicality, graded membership, and vagueness. *Cognitive Science*, 31(3): 355–384, 2007. doi: 10.1080/15326900701326402.

[17] James A. Hampton. Overextension of conjunctive concepts: Evidence for a unitary model of concept typicality and class inclusion. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 14(1):12–32, 1988. doi: 10.1037/0278-7393.14.1.12.

[18] Sharon Lee Armstrong, Lila R. Gleitman, and Henry Gleitman. What some concepts might not be. *Cognition*, 13(3):263–308, 1983. doi: 10.1016/0010-0277(83)90012-4.

[19] Allan M. Collins and Elizabeth F. Loftus. A spreading-activation theory of semantic processing. *Psychological Review*, 82(6):407–428, 1975. doi: 10.1037/0033-295X.82.6.407.

[20] John R. Anderson. *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press, New York, 2007.

[21] John H. Holland, Keith J. Holyoak, Richard E. Nisbett, and Paul R. Thagard. *Induction: Processes of Inference, Learning, and Discovery*. MIT Press, Cambridge, MA, 1986. doi: 10.7551/mitpress/3729.001.0001.

[22] David Klahr and Kevin Dunbar. Dual space search during scientific reasoning. *Cognitive Science*, 12(1):1–48, 1988. doi: 10.1207/s15516709cog1201_1.

[23] Abraham S. Luchins. Mechanization in problem solving: The effect of Einstellung. *Psychological Monographs*, 54(6):i–95, 1942. doi: 10.1037/h0093502.

[24] Paul Thagard. *Conceptual Revolutions*. Princeton University Press, Princeton, NJ, 1992. doi: 10.1515/9780691186672.

[25] John E. Laird. *The Soar Cognitive Architecture*. MIT Press, Cambridge, MA, 2012.

[26] John E. Laird, Christian Lebiere, and Paul S. Rosenbloom. A standard model of the mind: Toward a common computational framework across artificial intelligence, cognitive science, neuroscience, and robotics. *AI Magazine*, 38(4):13–26, 2017. doi: 10.1609/aimag.v38i4.2744.

Claim	Lean name (file)
The view is kept; the view wins on matched frames; unmatched frames survive; instances re-parented	`merge_keeps_view`, `merge_view_wins`, `merge_keeps_unmatched`, `merge_reparents` (`CogLib/Process/ViewApplication`)
BigTrak Table 4 reproduced	`table4_reproduced` (`ViewApplication/BigTrak`)
ARITHMETIC view: exact membership; no over-extension; typicality unchanged; dissociation	`after_exact`, `after_no_overextension`, `typicality_unchanged`, `dissociation` (`ViewApplication/Numbers`)
Selection by spreading activation	`naive`, `primed` (`ViewApplication/Selection`)
A confused cue brings in its neighbour's view; one view per cue for a determinate memory	`confusion_gives_neighbours_view`, `view_of_determinate` (`CogLib/Process/ViewRetrieval`)
The confusion-error pattern for view retrieval	`view_confusion_pattern`
An impasse is a failed prediction; a correct theory is never revised	`impasse_iff_fails`, `correct_theory_stable` (`CogLib/Process/TheoryRevision`)
The view reviser: its theories; memory fixed; perseveration for determinate memories	`viewReviser_theory`, `viewReviser_mem`, `revise_same_of_determinate` (`CogLib/Process/ViewRevision`)
Predictions depend only on the actions' function slots	`predict_congr` (`CogLib/Process/TheoryExecution`)
Garden path; perseveration; oscillation	`gardenPath`, `perseverates`, `oscillates` (`CogLib/Phenomena/TheoryChange`)
A view keeps what it does not mention	`merge_keeps_unmentioned` (`CogLib/Process/ViewComposition`)
A split changes no prediction; it separates; one type, one reading	`split_conservative`, `split_separates`, `same_parent_same_reading` (`CogLib/Process/TypeSplit`)
Microwave: no theory with a shared type predicts both keys	`never_both_shared` (`DeviceLearning/Microwave`)
Microwave: composing views; sessions with and without splitting	`one_view_not_enough`, `two_views_compose`, `revision_keeps_number`, `no_split_session`, `split_session` (`DeviceLearning/Differentiation`)
Microwave: memory facts; predictions of the views	`net_facts`, `act_graded`, `view_predictions` (`DeviceLearning/Session`)
Microwave: the pattern; perseveration; sessions; a correct theory kept	`epam_confusion_pattern`, `epam_revisions_same`, `epam_two_cues`, `act_two_cues`, `epam_label_only`, `act_label_only`, `duration_kept`

A Index of results

B The illustrative setting and exact values

Codes. Cues are coded by five features: a time word, a cooking word, a clock word, a counting-down display, and a colon.

- TIME COOK has the time and cooking words; CLOCK has the time and clock words; the cooking display has the countdown and the colon.
- The VCR's SET TIME has the time word, the clock word and a colon; the kitchen timer has the time word, the countdown and a colon.

EPAM's net learns the two prior items, noticing the countdown first, then the colon, then the words. ACT-R's memory studies them in the same order, with a mismatch penalty of 1/2 over all five features and a failure weight of 1/10. The TIME COOK label calls up the VCR with probability 5/23, the timer with 10/23 (equally similar, the timer more recent), and nothing with 8/23 (`act_graded`).

Sessions. The episode is TIME COOK 9 0 START, observed to cook for 90 seconds. Each session runs for at most four decision cycles. "Solved" means the final theory predicts the episode.

Learner	Garden path	Solved	Perseverates
EPAM, label and display	1/2	1	0
ACT-R, label and display	$125/1219 \approx .10$	$1050/1219 \approx .86$	$25/1219 \approx .02$
EPAM, label only	0	0	1
ACT-R, label only	$67450/279841 \approx .24$	$251280/279841 \approx .90$	$27225/279841 \approx .10$

Table 3: Exact values (`epam_two_cues`, `act_two_cues`, `epam_label_only`, `act_label_only`). The garden-path and perseveration columns are computed over the interpretations brought in, ignoring retrievals that called up nothing.

Decomposed views and splitting. A third prior item, a calculator keypad, is stored with SET-A-NUMBER; the digit keys are a cue for it, and EPAM's net notices digits first. The episodes are TIME COOK 9 0 START, CLOCK 9 0 START, and TIME COOK 9 0 START again; sessions run for at most twelve decision cycles.

Learner	Knows both keys	Finishes	Oscillates
Cannot split	0	5/6	5/6
Splits on conflict	5/6	5/6	0

Table 4: Exact values (`no_split_session`, `split_session`). In both, the 1/6 that do not finish brought in a wrong meaning last on the first episode and ran out of cues. Every splitting learner that finishes knows both keys.

These values are consequences of the chosen setting, not predictions about people.